Problem 1 - Persistent Vertices

Let $G = (V, E)$ be a finite directed graph (without self-loops). Call a vertex v *persistent* if there exists an infinite directed walk starting at v. Vertices and edges may be repeated in a walk.

(a) Characterization

Claim. v is persistent if and only if v can reach a vertex that lies on a directed cycle.

($\Rightarrow$) Let $W = (v_0, v_1, v_2, \ldots)$ be an infinite walk with $v_0 = v$. Since $|V|$ is finite, some vertex repeats: $v_i = v_j$ for some $i < j$. Then $(v_i, v_{i+1}, \ldots, v_j)$ is a directed cycle and $(v, \ldots, v_i)$ is a directed path from v into it.

($\Leftarrow$) Suppose $v \to \ldots \to u$ and u lies on a directed cycle C. Walk from v to u, then go around C forever: every step is legal, so the walk never stops.

Since G has no self-loops, every directed cycle has at least two vertices, so the vertices lying on cycles are exactly the vertices of strongly connected components (SCCs) of size ≥ 2. Therefore: ***v* is persistent iff *v* can reach an SCC of size ≥ 2.**

(b) $O(n + m)$-time algorithm

Idea: compute the SCCs (Kosaraju), mark the vertices of SCCs of size ≥ 2, and then traverse the **reversed** graph from all marked vertices; exactly the vertices reaching a cycle are reached.

```
def persistent_vertices(G):
    order = dfs_finishing_order(G)          # DFS on G        O(n+m)
    G_r   = reverse_graph(G)                #                 O(n+m)
    comp, sccs = [-1]*n, []
    for v in reversed(order):               # DFS on G_r      O(n+m)
        if comp[v] == -1:
            c = len(sccs); comp[v] = c
            stack, members = [v], []
            while stack:
                u = stack.pop(); members.append(u)
                for w in G_r.out(u):
                    if comp[w] == -1:
                        comp[w] = c; stack.append(w)
            sccs.append(members)

    persistent, stack = [False]*n, []
    for members in sccs:
        if len(members) >= 2:               # no self-loops => cyclic
            for v in members:
                persistent[v] = True; stack.append(v)

    while stack:                            # DFS on G_r from cycles
        u = stack.pop()
        for w in G_r.out(u):                # w -> u in G
            if not persistent[w]:
                persistent[w] = True; stack.append(w)
    return persistent
```

Correctness:

1. Kosaraju's algorithm returns exactly the SCCs of G (proved in lecture), so no cyclic vertex is missed and no acyclic one is marked.
2. By (a), the vertices of SCCs of size ≥ 2 are precisely the vertices on cycles.

3. Traversing G_r from those vertices reaches w iff G_r has a path from a cyclic vertex to w, iff G has a path $w \to \ldots \to$ (cyclic vertex). By (a) those and only those w are persistent. The array `persistent` also serves as the visited array, so every vertex is pushed at most once (and cyclic vertices are pushed once at the start).

Time complexity: both DFS passes, the graph reversal and the final traversal touch each vertex and each edge of G (resp. G_r) a constant number of times, so each is $O(n+m)$ and the total is $O(n+m)$. Note that all traversals use an explicit stack: using a Python list as a queue (`pop(0)`) costs $O(n)$ per deletion and would destroy the bound; a `collections.deque` or a stack are $O(1)$.

(c) The out-degree claim is false

Counterexample. $V = \{A, B, C\}$, $E = \{(A, B), (B, C)\}$. Vertex A has out-degree 1, but the graph is a DAG, so no cycle is reachable from A and the walk $A \to B \to C$ is stuck at C; A is not persistent.

The error. The implication "persistent $\Rightarrow$ out-degree > 0" is true: a persistent vertex either lies on a cycle or has an outgoing edge towards one. The converse fails: a positive out-degree only guarantees that the walk can be extended **one** more step, not that it can be extended forever. To walk forever one must revisit a vertex and thus move around a cycle; reachability of a cycle is a global property that the out-degree of a single vertex cannot express.

Problem 2 - Greedy

There are n jobs, each requiring one unit of processing time on a single machine. Job i has an integer deadline d_i with $1 \le d_i \le n$ and profit $p_i > 0$. A job earns its profit only if it finishes by its deadline, and a subset of jobs is scheduled in unit-length slots. Number the slots $1, \ldots, n$, where slot t finishes at time t; a job with deadline d may use any slot $t \le d$.

Algorithm: process jobs in nonincreasing order of profit, place each job in the latest currently empty slot that finishes by its deadline, discard it if there is none.

(a) The algorithm maximizes the total profit

Order the jobs $1, \ldots, n$ by nonincreasing profit (the order used by the algorithm), write $P_k = \{1, \ldots, k\}$, let G_k be the set accepted after the first k jobs, and let $N_S(t) = |\{i \in S : d_i \le t\}|$.

Fact 1 (feasibility). S is schedulable iff $N_S(t) \le t$ for every t.

If $N_S(t) > t$, those $N_S(t)$ jobs would have to share the t slots $1..t$. Conversely, if the condition holds, schedule S in nondecreasing order of deadline: the k-th job of this order receives slot k, and $k \le N_S(d) \le d$ for its deadline d.

Fact 2 (exchange). If S, T are schedulable and $|S| < |T|$, then some $j \in T \setminus S$ has $S \cup \{j\}$ schedulable.

Let $\tau = \max(\{0\} \cup \{t : N_S(t) = t\})$. By Fact 1, $S \cup \{j\}$ is schedulable iff $d_j > \tau$: if $d_j \le \tau$ then $N_{S\cup\{j\}}(\tau) = \tau + 1$, and if $d_j > \tau$ then $N_S(t) \le t - 1$ for every $t \ge d_j$ while all $t < d_j$ are unaffected. If every $j \in T \setminus S$ had $d_j \le \tau$, then, as no job of T with deadline $> \tau$ lies outside S,

$$|T| = N_T(\tau) + |\{j \in T : d_j > \tau\}| \le \tau + |\{j \in S : d_j > \tau\}| = N_S(\tau) + |\{j \in S : d_j > \tau\}| = |S|,$$

contradicting $|S| < |T|$.

Fact 3. $|G_k| \ge |B|$ for every schedulable $B \subseteq P_k$.

The algorithm rejects j only when the job cannot fit: if slots $1, \ldots, d_j$ are all occupied and $r \geq d_j$ is the longest fully occupied prefix, then every one of the r jobs a in slots $1..r$ has $d_a \leq r$ — if $d_a \geq r+1$ (impossible when $r = n$), slot $r+1$ was free both at a's insertion and at the end, so "latest free slot $\leq d_a$" would have chosen it. Hence $N_{G_{j-1}}(r) = r$ and, since $d_j \leq r$, Fact 1 shows that $G_{j-1} \cup \{j\}$ is not schedulable. Now if some schedulable $B \subseteq P_k$ had $|B| > |G_k|$, Fact 2 would give $j \in B \setminus G_k$ with $G_k \cup \{j\}$ schedulable; but $G_{j-1} \cup \{j\} \subseteq G_k \cup \{j\}$ and subsets of schedulable sets are schedulable, a contradiction.

Conclusion. So for every schedulable O and every k, $|G \cap P_k| = |G_k| \geq |O \cap P_k|$. With $p_{n+1} = 0$ and $p_j = \sum_{k=j}^{n}(p_k - p_{k+1})$ we have $w(S) = \sum_{k=1}^{n}(p_k - p_{k+1})\ |S \cap P_k|$ for any job set S, hence

$$w(G) - w(O) = \sum_{k=1}^{n}(p_k - p_{k+1})(|G \cap P_k| - |O \cap P_k|) \geq 0$$

because every profit difference is ≥ 0 and $p_n > 0$; the greedy profit is optimal. ∎

In words: greedy keeps the maximum **number** of accepted jobs in every profit prefix, and since the profits decrease, that prefix-wise dominance together with Abel summation already makes the greedy profit unbeatable.

(b) $O(n^2)$-time implementation

```
def schedule(jobs):                 # jobs[i] = (profit, deadline)
    jobs.sort(key=lambda j: j.profit, reverse=True)     # O(n log n)
    n = len(jobs)
    slot = [None] * (n + 1)         # slot[t] = job finishing at time t
    for j in jobs:
        # latest free slot that finishes by the deadline
        for t in range(min(j.deadline, n), 0, -1):
            if slot[t] is None:
                slot[t] = j
                break               # else the job is discarded
    return [(t, slot[t]) for t in range(1, n + 1)
            if slot[t] is not None]
```

Correctness: the returned list is exactly the set of accepted jobs together with their slots, which is schedulable by construction (distinct slots, each job before its deadline), and it is optimal by part (a).

Time complexity: sorting costs $O(n \log n) \leq O(n^2)$. The outer loop runs n times and the inner loop examines at most n slots per job, so the scheduling phase is $O(n^2)$. Total $O(n^2)$. (Scanning the deadlines of the remaining jobs is unnecessary because $d_i \leq n$; replacing the inner loop by a union-find "next free slot" structure gives $O(n \log n)$.)

Problem 3 - Counting Shortest Paths

$G = (V, E)$ is a directed graph with strictly positive edge weights $w(e) > 0$, source s, and integer $M \geq 2$. For every vertex v we output $\mathrm{dist}(v)$ (length of a shortest $s \to v$ path) and $\mathrm{cnt}(v)$ (number of distinct shortest $s \to v$ paths, modulo M).

Algorithm: Dijkstra with an **indexed** priority queue, i.e. a min-heap that supports DECREASE-KEY in $O(\log n)$ — a binary heap plus a position array `pos[v]` holding the current heap index of v. Consequences: every vertex is inserted exactly once and never re-inserted, so the heap never contains stale entries, and each edge relaxation triggers at most one DECREASE-KEY.

```
def count_shortest_paths(G, s, M):
    dist = [INF] * n
```

```
    cnt = [0] * n
    dist[s] = 0
    cnt[s] = 1
    H = IndexedMinHeap(dist)  # indexed heap, uses pos[v]; build O(n)
    while len(H) > 0:
        u = H.delete_min()    # dist[u], cnt[u] are now final
        if dist[u] == INF: break          # the rest is unreachable
        for (v, w) in G.out(u):           # each edge relaxed once
            nd = dist[u] + w
            if nd < dist[v]:
                dist[v] = nd
                cnt[v] = cnt[u]   # old (longer) paths are discarded
                H.decrease_key(v, nd)     # O(log n) via pos[v]
            elif nd == dist[v]:
                cnt[v] = (cnt[v] + cnt[u]) % M   # one more path
    return dist, cnt
```

Why the `elif` branch is safe. If the equality $\mathrm{nd} = \mathrm{dist}[v]$ held while v had already been deleted from H, then $\mathrm{dist}[v] \le \mathrm{dist}[u]$ (keys are deleted in nondecreasing order) and $\mathrm{dist}[u] = \mathrm{dist}[v] - w < \mathrm{dist}[v]$ because $w > 0$ — a contradiction. So v is still in the heap and its count is not yet final.

Correctness:

Invariant. After every operation, for each v: $\mathrm{dist}[v]$ is the length of a shortest $s \to v$ path found so far (∞ if none), and $\mathrm{cnt}[v]$ is the number of distinct $s \to v$ paths of that length, modulo M.

Relaxing (u, v) of weight w: if $\mathrm{nd} < \mathrm{dist}[v]$, all paths counted by $\mathrm{cnt}[v]$ are longer than nd and are discarded, and every new path of length nd ends with the edge (u, v) and corresponds bijectively to an $s \to u$ path of length $\mathrm{dist}[u]$ (the prefix is shorter because $w > 0$), so the new count is $\mathrm{cnt}[u]$; if $\mathrm{nd} = \mathrm{dist}[v]$, the shortest paths split into those not using (u, v) and those using it, giving $\mathrm{cnt}[v] + \mathrm{cnt}[u]$.

Finality. When u is deleted, all heap keys are $\ge \mathrm{dist}[u]$ and every edge leaving the settled set has positive weight, so no shorter $s \to u$ path can appear later: standard Dijkstra exchange argument. For the count, every shortest path to u ends with an edge (x, u) where $\mathrm{dist}[x] = \mathrm{dist}[u] - w(x, u) < \mathrm{dist}[u]$, so x was deleted earlier and the edge (x, u) was relaxed at that moment. By induction on the deletion order, $\mathrm{cnt}[u]$ is final when u is deleted. Since a path counting is additive and $(A + B) \bmod M = ((A \bmod M) + (B \bmod M)) \bmod M$, reducing modulo M after each addition gives the same result as reducing once at the end.

Time complexity: building the heap costs $O(n)$; there are exactly n DELETE-MIN operations ($n \log n$ total); at most one DECREASE-KEY is triggered per relaxed edge, hence at most m of them, each $O(\log n)$ thanks to the position array; the equal-distance updates are $O(1)$. Total $O((n + m) \log n)$ and $O(n + m)$ space. No lazy duplicates occur, so the accounting is tight — this is where an unindexed heap with lazy deletion would lose the $O((n + m) \log n)$ guarantee when multi-edges are allowed.

Problem 4 - Minimum Spanning Tree with Required Edges

$G = (V, E)$ is a weighted undirected graph, $R \subseteq E$ is a forest that must be contained in the spanning tree. We assume a simple graph, so $m \le n^2$ and $\log m = O(\log n)$ (if parallel edges are allowed, keep only the lightest edge of each pair in $O(m)$ expected time with hashing, which preserves the bound).

```python
def constrained_mst(G, R):
    n, m = G.n, G.m
    if n > 1 and m < n - 1: return None          # no spanning tree at all
    uf, T = UnionFind(n), set()
    for (u, v) in R:                             # contract the forest R
        if uf.find(u) == uf.find(v): return "R is not a forest"
        uf.union(u, v); T.add((u, v))
    for (u, v, w) in sorted(G.edges - R, key=w): # Kruskal on E - R
        if len(T) == n - 1: break
        if uf.find(u) != uf.find(v):
            T.add((u, v)); uf.union(u, v)
    return T if len(T) == n - 1 else None
```

Correctness:

1. **No cycle is ever created.** R is a forest, so pre-loading it into T adds no cycle, and in the Kruskal phase an edge is added only when its endpoints lie in different components, so no cycle is added either (and `R`, being given, is not modified).
2. **Valid output.** If the algorithm ends with $|T| = n - 1$, then T is acyclic and connected on n vertices, i.e. a spanning tree that contains R.
3. **`None` is correct.** If $|T| < n - 1$ at the end, every edge of E has been scanned and the union-find still has more than one component; since $R \subseteq E$, the components of the union-find are exactly the connected components of G, so G is disconnected and no spanning tree (containing R or not) exists. The early check $m < n - 1$ (for $n > 1$) reports the same obstruction immediately.
4. **Optimality.** Contract every tree of the forest R, i.e. every connected component of (V, R), into one super-vertex and delete self-loops, keeping the lightest edge between each pair of super-vertices. This gives a multigraph G/R. A set $T \supseteq R$ is a spanning tree of G iff $T \setminus R$ is a spanning tree of G/R: both conditions say "each contraction class is connected by the chosen edges and the chosen edges are acyclic". So minimizing $w(T)$ over spanning trees containing R is exactly minimizing the weight of a spanning tree of G/R with the edges $E \setminus R$. Our algorithm does precisely that: the initial unions of R build the super-vertices, and the scan of $E \setminus R$ in nondecreasing weight with the Kruskal test is exactly Kruskal's algorithm on G/R, which returns an MST of G/R. Hence the output is a minimum-weight spanning tree containing R.

Time complexity ($n = |V|$, $m = |E|$):

1. Union-find initialization: $O(n)$. Processing R: at most $|R| \leq n - 1$ find/union pairs, $O(n\alpha(n))$.
2. Sorting $E \setminus R$: at most m edges, $O(m \log m) = O(m \log n)$ since $m \leq n^2$; a Fibonacci/binary heap-based Prim implementation would give the same bound.
3. Kruskal scan: at most m iterations, each two finds plus at most one union, $O(m\alpha(n)) = O(m \log n)$.
4. Because $m \geq n - 1$ (the other case is handled by the test in the first line, and $n \leq 2$ is trivial), $n = O(m)$, so $O(n\alpha(n)) \subseteq O(m \log n)$.

Total: $O(m \log n)$ time, $O(n + m)$ space.